03 — Transaction Isolation Levels

Table of Contents

Learning Objectives

After studying this file you will be able to:

- Define all four SQL standard isolation levels and the anomalies each prevents
- Explain how PostgreSQL implements each level using MVCC
- Reproduce dirty read, non-repeatable read, phantom read, and serialization anomaly
- Choose the appropriate isolation level for a given workload
- Configure isolation levels per session and per transaction
- Understand PostgreSQL's deviation from the SQL standard for READ UNCOMMITTED

The Problem: Concurrency Anomalies

When multiple transactions run concurrently, they can interfere in ways that produce incorrect results. The SQL standard defines four **anomalies** that isolation levels protect against:

Dirty Read

Transaction A reads data written by Transaction B that has NOT yet committed. If B rolls back, A has read data that never existed.

```
Txn A:            BEGIN; UPDATE accounts SET bal=0 WHERE id=1;
Txn B:   BEGIN;   READ accounts WHERE id=1  --> sees bal=0 (dirty!)
Txn A:   ROLLBACK;
Txn B:   -- Used bal=0 in a calculation -- WRONG
```

Non-Repeatable Read

Transaction A reads a row. Transaction B updates and commits that row. Transaction A reads the same row again and gets a different result.

```
Txn A:  BEGIN; SELECT price FROM products WHERE id=1;  --> 100
Txn B:           UPDATE products SET price=200 WHERE id=1; COMMIT;
Txn A:           SELECT price FROM products WHERE id=1;  --> 200 (different!)
```

Phantom Read

Transaction A reads a set of rows matching a condition. Transaction B inserts new rows matching that condition and commits. Transaction A re-reads the condition and sees additional "phantom" rows.

```
Txn A:  BEGIN; SELECT COUNT(*) FROM orders WHERE status='pending';  --> 5
Txn B:           INSERT INTO orders(status) VALUES('pending'); COMMIT;
Txn A:           SELECT COUNT(*) FROM orders WHERE status='pending';  --> 6 (phantom!)
```

Serialization Anomaly (Write Skew)

Two transactions each read overlapping data and make decisions based on what they read, producing a result that could not have occurred if the transactions had run serially.

```
Constraint: at least one doctor must be on-call

Txn A:  SELECT COUNT(*) FROM doctors WHERE on_call=true;  --> 2
Txn B:  SELECT COUNT(*) FROM doctors WHERE on_call=true;  --> 2
Txn A:  UPDATE doctors SET on_call=false WHERE id=1;
Txn B:  UPDATE doctors SET on_call=false WHERE id=2;
Both commit --> 0 doctors on call!  Constraint violated.
```

SQL Standard Isolation Levels

The SQL standard defines four isolation levels as the minimum guarantees:

```
+--------------------+-------------+-------------------+--------------+---------------
-------+
| Isolation Level    | Dirty Read  | Non-Repeatable    | Phantom Read | Serialization
Anomaly|
+--------------------+-------------+-------------------+--------------+---------------
-------+
| Read Uncommitted   | Possible    | Possible          | Possible     | Possible
|
| Read Committed     | Not Possible| Possible          | Possible     | Possible
|
| Repeatable Read    | Not Possible| Not Possible      | Possible*    | Possible
|
| Serializable       | Not Possible| Not Possible      | Not Possible | Not Possible
|
+--------------------+-------------+-------------------+--------------+---------------
-------+
* SQL standard allows phantoms; PostgreSQL REPEATABLE READ also prevents them
```

Anomaly Reference Table

PostgreSQL's actual behavior (stricter than the SQL standard in some places):

```
+--------------------+-------------+-------------------+--------------+---------------
-------+
| PG Isolation Level | Dirty Read  | Non-Repeatable    | Phantom Read | Serialization
Anomaly|
+--------------------+-------------+-------------------+--------------+---------------
-------+
| Read Uncommitted   | Not Possible| Possible          | Possible     | Possible
|
| Read Committed     | Not Possible| Possible          | Possible     | Possible
|
| Repeatable Read    | Not Possible| Not Possible      | Not Possible | Possible
|
| Serializable       | Not Possible| Not Possible      | Not Possible | Not Possible
|
+--------------------+-------------+-------------------+--------------+---------------
-------+

Note: PostgreSQL READ UNCOMMITTED behaves the same as READ COMMITTED.
      PostgreSQL REPEATABLE READ prevents phantoms (stronger than SQL standard).
```

Read Uncommitted

SQL Standard: Allows dirty reads, non-repeatable reads, and phantom reads.

PostgreSQL Behavior: PostgreSQL does NOT implement true Read Uncommitted. Due to MVCC, even at this level, a transaction will only see committed data. PostgreSQL treats `READ UNCOMMITTED` as `READ COMMITTED`.

```
-- These are equivalent in PostgreSQL:
BEGIN TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Use case: There is no reason to use READ UNCOMMITTED in PostgreSQL.

Read Committed (PostgreSQL Default)

Snapshot acquisition: New snapshot at the start of each SQL statement.

Prevents: Dirty reads. **Allows:** Non-repeatable reads, phantom reads.

Behavior

Each statement sees all data committed before that statement began. Two SELECT statements in the same transaction can return different results if another transaction commits between them.

```
-- Session A:
BEGIN;
SELECT balance FROM accounts WHERE id=1;   -- returns 1000
-- (Session B: UPDATE accounts SET balance=500 WHERE id=1; COMMIT;)
SELECT balance FROM accounts WHERE id=1;   -- returns 500! (non-repeatable read)
COMMIT;
```

When to Use

- Default for most OLTP applications
- When you want fresh data on each statement
- When the application handles the possibility of read results changing between statements

Lost Update Problem Under Read Committed

```
-- Classic race condition even with READ COMMITTED:
-- Txn A:                          Txn B:
BEGIN;                             BEGIN;
SELECT val FROM t WHERE id=1; -- 10    SELECT val FROM t WHERE id=1; -- 10
                                   UPDATE t SET val=10+5 WHERE id=1;
                                   COMMIT; -- val=15
UPDATE t SET val=10+3 WHERE id=1;
COMMIT; -- val=13 !! Lost B's update
```

Fix: Use `SELECT ... FOR UPDATE` to lock the row, or use atomic updates (`UPDATE t SET val=val+3`).

Repeatable Read

Snapshot acquisition: Once at the start of the first statement; reused for the entire transaction.

Prevents: Dirty reads, non-repeatable reads, phantom reads (PostgreSQL-specific — stricter than standard).
Allows: Serialization anomalies (write skew).

Behavior

The transaction sees a consistent snapshot of the entire database as it existed when the transaction began its first statement. Even if other transactions commit changes, this transaction always sees the same data.

```
-- Session A:
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id=1;   -- returns 1000
-- (Session B: UPDATE accounts SET balance=500 WHERE id=1; COMMIT;)
SELECT balance FROM accounts WHERE id=1;   -- STILL returns 1000 (repeatable!)
COMMIT;
```

Serialization Error on Conflicting Updates

Under REPEATABLE READ, if two transactions try to update the same row, the second one to commit gets an error:

```
-- Txn A:                              Txn B:
BEGIN ISOLATION LEVEL REPEATABLE READ;  BEGIN ISOLATION LEVEL REPEATABLE READ;
UPDATE accounts SET bal=bal-100         UPDATE accounts SET bal=bal+50
  WHERE id=1;                             WHERE id=1;
COMMIT;  -- succeeds                    COMMIT;
-- ERROR: could not serialize access due to concurrent update
-- Application must RETRY Txn B
```

When to Use

- Reports that need a consistent view of the database across multiple queries
- Multi-step read operations where the data must not change mid-process
- Aggregations across multiple related tables that must be consistent with each other

---

Serializable

Snapshot acquisition: Once at start; uses SSI (Serializable Snapshot Isolation) predicate tracking.

Prevents: All anomalies including serialization anomalies (write skew, read-only transaction anomalies).
Behavior: The result of concurrent transactions is equivalent to some serial execution of those transactions.

```
-- The on-call doctor example now works correctly:
-- Txn A:
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors WHERE on_call=true;  -- 2
UPDATE doctors SET on_call=false WHERE id=1;

-- Txn B (concurrent):
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors WHERE on_call=true;  -- 2
UPDATE doctors SET on_call=false WHERE id=2;
COMMIT;  -- succeeds (first one in)

-- Txn A COMMIT:
-- ERROR: could not serialize access due to read/write dependencies among
transactions
-- Application RETRIES Txn A
```

How PostgreSQL Implements Serializable

PostgreSQL uses **Serializable Snapshot Isolation (SSI)** — see `07_serializable_snapshot_isolation.md`. It tracks read-write dependencies between transactions using predicate locks and detects dangerous cycles.

Performance Cost of Serializable

- Predicate lock overhead (memory and CPU)
- More frequent serialization errors requiring application retry logic
- Slightly higher transaction latency

Typically 1.5x–3x overhead vs READ COMMITTED for write-heavy workloads.

When to Use

- Financial systems where any inconsistency is unacceptable
- Inventory systems preventing double-booking
- Any system where business correctness requires true isolation
- Systems where the logic is complex enough to produce write skew

---

Setting Isolation Levels

Per Transaction

```
-- Method 1: In the BEGIN command
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

-- Method 2: SET TRANSACTION (must be first statement after BEGIN)
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT ...;
COMMIT;

-- Method 3: SET TRANSACTION shorthand
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Per Session

```
-- Set default for the current session
SET default_transaction_isolation = 'repeatable read';
SET default_transaction_isolation = 'serializable';
SET default_transaction_isolation = 'read committed';
```

Per Role or Database

```
-- For all connections as a specific user:
ALTER ROLE analytics_user SET default_transaction_isolation = 'repeatable read';

-- For all connections to a database:
ALTER DATABASE reporting SET default_transaction_isolation = 'repeatable read';
```

System-Wide Default

```
-- In postgresql.conf:
-- default_transaction_isolation = 'read committed'
```

Check Current Level

```
SHOW transaction_isolation;
SHOW default_transaction_isolation;
```

Choosing the Right Level

```
Decision Tree:
                     Need reads to be
                     fully consistent?
                     /           \
                   NO             YES
                   |               |
            Need to prevent        Need to prevent
            dirty reads?           write skew?
            (Yes by default)       /         \
                 |               NO           YES
                 |               |             |
           READ COMMITTED    REPEATABLE      SERIALIZABLE
           (default)         READ
```

Quick Reference

Workload	Recommended Level
Standard OLTP (reads + writes)	READ COMMITTED
Consistent reports across tables	REPEATABLE READ
Financial batch processing	SERIALIZABLE
Background analytics	REPEATABLE READ
Inventory / booking systems	SERIALIZABLE
Simple CRUD	READ COMMITTED

Anomaly Demonstrations

Demonstrating Non-Repeatable Read (READ COMMITTED)

```
-- Terminal 1:
BEGIN;
SELECT salary FROM employees WHERE id = 1;  -- 50000
-- PAUSE -- let Terminal 2 run

-- Terminal 2:
BEGIN;
UPDATE employees SET salary = 60000 WHERE id = 1;
```

```
COMMIT;

-- Terminal 1 (continue):
SELECT salary FROM employees WHERE id = 1;  -- 60000 (non-repeatable!)
COMMIT;
```

Demonstrating Prevention (REPEATABLE READ)

```
-- Terminal 1:
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT salary FROM employees WHERE id = 1;  -- 50000
-- PAUSE -- let Terminal 2 run

-- Terminal 2:
BEGIN;
UPDATE employees SET salary = 60000 WHERE id = 1;
COMMIT;

-- Terminal 1 (continue):
SELECT salary FROM employees WHERE id = 1;  -- STILL 50000
COMMIT;
```

Demonstrating Write Skew (REPEATABLE READ)

```
-- Setup: Both rows in same group must have same active status
CREATE TABLE schedule (
  id int PRIMARY KEY,
  on_call bool,
  name text
);
INSERT INTO schedule VALUES (1, true, 'Alice'), (2, true, 'Bob');

-- Terminal 1:
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM schedule WHERE on_call = true;  -- 2, ok to remove one
UPDATE schedule SET on_call = false WHERE id = 1;

-- Terminal 2 (concurrent):
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT COUNT(*) FROM schedule WHERE on_call = true;  -- 2, ok to remove one
UPDATE schedule SET on_call = false WHERE id = 2;
COMMIT;  -- succeeds

-- Terminal 1:
COMMIT;  -- succeeds -- but now BOTH are off-call: constraint violated!
-- REPEATABLE READ does NOT prevent write skew
```

Preventing Write Skew with SERIALIZABLE

```
-- Terminal 1:
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM schedule WHERE on_call = true;
UPDATE schedule SET on_call = false WHERE id = 1;
COMMIT;  -- succeeds

-- Terminal 2 (concurrent):
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM schedule WHERE on_call = true;
UPDATE schedule SET on_call = false WHERE id = 2;
COMMIT;  -- ERROR: could not serialize access due to read/write dependencies
-- Application retries and detects only 1 on-call, aborts the update
```

Common Mistakes

1. Using READ COMMITTED for Consistent Reports

```
-- BAD: balance sheet totals may be inconsistent
BEGIN;  -- READ COMMITTED
SELECT SUM(balance) FROM assets;     -- snapshot at T1
-- Other txn runs, modifies data
SELECT SUM(balance) FROM liabilities; -- snapshot at T2 (different!)
COMMIT;

-- GOOD:
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT SUM(balance) FROM assets;
SELECT SUM(balance) FROM liabilities;  -- same snapshot
COMMIT;
```

2. Not Retrying Serialization Errors

```
-- Serializable and Repeatable Read can produce:
-- ERROR: could not serialize access due to concurrent update
-- Application MUST detect this error code (40001 for serialization_failure,
-- 40P01 for deadlock_detected) and retry the entire transaction.
```

3. Mixing Isolation Levels in the Same Logical Operation

Comparing data read at READ COMMITTED in one connection with data from REPEATABLE READ in another can produce logically inconsistent results.

4. Believing READ UNCOMMITTED Is Useful in PostgreSQL

PostgreSQL does not support true dirty reads. READ UNCOMMITTED is identical to READ COMMITTED. Do not use it hoping for performance benefits.

Best Practices

1. **Keep READ COMMITTED as default** — override per-transaction where stricter semantics are needed.
2. **Use REPEATABLE READ for analytics** — ensures consistent cross-table reads.
3. **Use SERIALIZABLE for critical business logic** — implement retry loops in the application.
4. **Implement retry logic for error codes 40001 and 40P01** — serialization failures and deadlocks.
5. **Set `lock_timeout`** even with MVCC — some operations (DDL, `SELECT FOR UPDATE`) still acquire locks.
6. **Document the isolation level** for each critical code path in your application.

Performance Considerations

Overhead per Level

```
READ COMMITTED:   Snapshot taken per statement. Fast. No retry logic needed.
REPEATABLE READ:  Snapshot taken once. Slight overhead for concurrent update
detection.
SERIALIZABLE:     SSI predicate lock tracking. 10-30% overhead; retry logic required.
```

Serialization Retry Pattern

```python
import psycopg2
from psycopg2 import errors

def run_serializable(conn, func):
    while True:
        try:
            with conn.cursor() as cur:
                cur.execute("BEGIN ISOLATION LEVEL SERIALIZABLE")
                result = func(cur)
                conn.commit()
                return result
        except errors.SerializationFailure:
            conn.rollback()
            # exponential backoff recommended
        except errors.DeadlockDetected:
            conn.rollback()
```

Monitoring Serialization Failures

```sql
-- Count serialization failures
SELECT sum(xact_rollbacks) FROM pg_stat_database;

-- More detail via pg_stat_bgwriter and application logging
```

Interview Questions and Answers

Q1. What are the four isolation levels defined by the SQL standard?

A: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE.

Q2. What is a dirty read and which isolation level prevents it?

A: A dirty read occurs when transaction A reads data written but not yet committed by transaction B. If B rolls back, A has read data that never existed. READ COMMITTED (and higher) prevents dirty reads.

Q3. How does PostgreSQL differ from the SQL standard for READ UNCOMMITTED?

A: The SQL standard allows dirty reads at READ UNCOMMITTED. PostgreSQL's MVCC architecture means even at READ UNCOMMITTED, only committed data is visible. PostgreSQL treats READ UNCOMMITTED identically to READ COMMITTED.

Q4. What is a phantom read? Does REPEATABLE READ in PostgreSQL prevent it?

A: A phantom read occurs when a re-executed range query returns different rows (new rows inserted by another committed transaction). The SQL standard says REPEATABLE READ allows phantoms. PostgreSQL's REPEATABLE READ prevents them because it uses a fixed snapshot for the entire transaction.

Q5. What is write skew and which isolation level prevents it?

A: Write skew is a serialization anomaly where two transactions each read overlapping data, make decisions, and produce a combined result that no serial execution could produce. Only SERIALIZABLE prevents write skew.

Q6. How does PostgreSQL implement Serializable isolation?

A: Through Serializable Snapshot Isolation (SSI). PostgreSQL takes an MVCC snapshot like REPEATABLE READ, but additionally tracks read-write dependencies between transactions using predicate locks. If a dangerous cycle is detected (indicating a non-serializable execution), one transaction is aborted with a serialization failure.

Q7. What error code should an application retry on for serialization failures?

A: SQLSTATE 40001 (`serialization_failure`). Also retry on 40P01 (`deadlock_detected`).

Q8. When is READ COMMITTED insufficient?

A: When you need: (1) consistent cross-table reads for reports, (2) prevention of non-repeatable reads, (3) prevention of lost updates without explicit locking, or (4) prevention of write skew.

Q9. What happens if you call SET TRANSACTION ISOLATION LEVEL after the first statement in a transaction?

A: PostgreSQL raises `ERROR: SET TRANSACTION ISOLATION LEVEL must be called before any query` . The isolation level must be set as the first statement after BEGIN, or in the BEGIN command itself.

Q10. What is the difference between a serialization failure and a deadlock?

A: A serialization failure (40001) means the transaction's operations cannot be part of any serial ordering alongside the other concurrent transactions — it is an MVCC/SSI concept. A deadlock (40P01) means two transactions are each waiting for the other to release a lock — it is a locking concept. Both require application retry.

Q11. Why might you use REPEATABLE READ for a long-running report instead of SERIALIZABLE?

A: REPEATABLE READ provides a consistent snapshot (preventing non-repeatable reads and phantoms) with lower overhead than SERIALIZABLE (no predicate lock tracking, fewer serialization errors). For read-only reports, SERIALIZABLE's extra protection against write skew is irrelevant.

Q12. How do you set the isolation level for all new connections from a specific user?

```
ALTER ROLE report_user SET default_transaction_isolation = 'repeatable read';
```

Exercises and Solutions

Exercise 1 — Reproduce Non-Repeatable Read

Task: Using two psql sessions, demonstrate a non-repeatable read under READ COMMITTED, then prevent it with REPEATABLE READ.

Solution:

```
-- Session 1 (READ COMMITTED):
BEGIN;
SELECT val FROM test_table WHERE id = 1;  -- note value

-- Session 2:
UPDATE test_table SET val = 'new_value' WHERE id = 1;
COMMIT;

-- Session 1 (continue):
SELECT val FROM test_table WHERE id = 1;  -- different value
COMMIT;

-- Session 1 (REPEATABLE READ):
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT val FROM test_table WHERE id = 1;  -- note value

-- Session 2: repeat UPDATE + COMMIT

-- Session 1:
SELECT val FROM test_table WHERE id = 1;  -- SAME value
COMMIT;
```

Exercise 2 — Write Skew Prevention

Task: Demonstrate write skew under REPEATABLE READ, then show SERIALIZABLE prevents it.

Setup:

```
CREATE TABLE call_schedule (id int PRIMARY KEY, on_call bool, name text);
INSERT INTO call_schedule VALUES (1, true, 'Alice'), (2, true, 'Bob');
```

See "Anomaly Demonstrations" section above.

Exercise 3 — Implement Retry Logic

Task: Write a PL/pgSQL function that runs under SERIALIZABLE and retries on serialization failure.

Solution:

```
CREATE OR REPLACE FUNCTION safe_transfer(from_id int, to_id int, amount numeric)
RETURNS void LANGUAGE plpgsql AS $$
DECLARE
  retries int := 0;
BEGIN
  LOOP
    BEGIN
      SET LOCAL default_transaction_isolation = 'serializable';
      UPDATE accounts SET balance = balance - amount WHERE id = from_id;
      UPDATE accounts SET balance = balance + amount WHERE id = to_id;
      RETURN;
    EXCEPTION
      WHEN serialization_failure OR deadlock_detected THEN
        retries := retries + 1;
        IF retries > 5 THEN
          RAISE;
        END IF;
        PERFORM pg_sleep(0.1 * retries);  -- exponential backoff
    END;
  END LOOP;
END;
$$;
```

Production Troubleshooting Scenarios

Scenario 1: Inconsistent Report Totals

Symptom: Balance sheet report shows assets != liabilities by small amounts that vary each run.

Cause: Report uses READ COMMITTED; multiple queries see different snapshots as data changes.

Fix:

```
-- Wrap entire report in REPEATABLE READ transaction
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ READ ONLY;
-- run all report queries here
COMMIT;
```

Scenario 2: High Serialization Failure Rate

Symptom: Under SERIALIZABLE, 30% of transactions fail with serialization_failure.

Diagnosis:

```sql
-- Check pg_stat_database for rollbacks
SELECT datname, xact_rollbacks, xact_commit,
       round(100.0 * xact_rollbacks / nullif(xact_commit + xact_rollbacks, 0), 2) AS
rollback_pct
FROM pg_stat_database
WHERE datname = current_database();
```

Fix:

- Shorten transactions to reduce conflict window
- Use `SELECT ... FOR UPDATE` to acquire locks early and deterministically
- Consider REPEATABLE READ if write skew is not actually a concern
- Implement exponential backoff in retry logic

Cross-References

- `02_mvcc.md` — MVCC snapshot mechanics that underpin all isolation levels
- `04_locks.md` — Lock-based mechanisms used alongside MVCC
- `07_serializable_snapshot_isolation.md` — Deep dive into SSI implementation
- `08_transaction_interview_guide.md` — All isolation level interview questions